CAMPAIGN INTEGRITY

Version-Aware Approval Architecture for Enterprise Marketing Systems

Track: Martech Approval | Team: MAL 4 | Date: 1 Nov 2026

Architecture-First Submission

Deliverable: One-Page Solution Brief

THE CORE ARCHITECTURAL INVARIANT

"The campaign that gets scheduled is the exact campaign that was approved."

```
active_schedule(c) ⇒ current_rev(c).state = APPROVED ∧ decision.rev_id = current_rev(c).id ∧ decision.content_hash = current_rev(c).content_hash
```

1 PROBLEM: DETACHED APPROVAL, NOT MISSING STATUS

Marketing platforms treat approval as a mutable status flag (status = "APPROVED") on the campaign container. When budget, audience, or channel is modified post-approval, that flag persists, silently launching unreviewed mutations.

1. Marketer Submits	2. Reviewer Approves	3. Silent Mutation	4. State Drift	5. Launch
Proposal: ₹5L budget & Tier-1 audience. →	Validates ₹5L budget intent. →	Budget altered to ₹8L post-approval. →	Badge stays "APPROVED". →	Unreviewed launch executes.

The approval was authentic, but authorised content that no longer exists.

2 USERS & SEPARATION OF DUTIES

ROLE	CORE RESPONSIBILITIES	ENFORCED RESTRICTIONS
Marketer	Drafts proposals, edits drafts, submits for review, revises after feedback, schedules approved campaigns.	**Cannot approve any campaign**; review role strictly separated.
Reviewer	Evaluates current revisions; approves or requests changes with mandatory written rationale.	**Cannot approve self-authored campaigns**; cannot edit; cannot approve stale revisions.
System	Enforces copy-on-write, checks SHA-256 hashes, executes row-locked scheduling checks, logs audit trail.	**Timeouts never treated as success**; derived state prevents boolean drift.

3 CORE REVISION LIFECYCLE (COPY-ON-WRITE)

1 **Draft Proposal** Rev 1 created in DRAFT.	2 **Submit Review** Rev 1 locked in IN_REVIEW.	3 **Feedback Given** CHANGES_REQUESTED with note.	4 **Revise & Fork** Rev 2 DRAFT created (Rev 1 superseded).
5 **Resubmission** Rev 2 placed in IN_REVIEW.	6 **Bound Approval** Rev 2 APPROVED + SHA-256 hash.	7 **Schedule Gate** SCHEDULED under row-lock check.	8 **Post-Edit Guard** Edit triggers Rev 3; cancels schedule!

Transactional Invariant: Any post-approval edit executes copy-on-write fork, revoking existing approval and automatically canceling active schedules within the same atomic PostgreSQL transaction.

4 ARCHITECTURAL BUSINESS RULES

- **BR-1** **Eligibility Gate:** Only campaigns in approved state are eligible for scheduling. Enforced by ScheduleService transaction.
- **BR-2** **Revision & Hash Bound:** Decision must explicitly reference current revision and match exact SHA-256 content hash.
- **BR-3** **Copy-on-Write Mutation:** Editing an approved proposal supersedes revision, revoking approval & active schedule atomically.
- **BR-4** **Materiality:** In v1, all fields (objective, audience, channel, budget in paise) are material. Any field edit spawns a new revision.
- **BR-5** **Command Idempotency:** Retries produce exactly one domain effect and audit event via client-generated UUID keys.
- **BR-6** **Self-Review Prohibited:** Author cannot review own proposal (separation of duties). Application service authorization check.

5 ARCHITECTURAL DEFENSIBILITY PILLARS

Single Atomic Gate
Scheduling validates revision state, hash, and schedule uniqueness within one PostgreSQL transaction (`SELECT FOR UPDATE`). Zero race windows.

Derived, Not Stored
Approval is dynamically derived from `(current_rev, active_decision, matching_hash)`. Eliminates mutable boolean status drift by design.

Crash & Network Proof
Client SQLite outbox + idempotency keys prevent duplicate taps and reconcile lost responses after app kill or sudden network drops.

Tamper-Evident Audit
Append-only audit trail logs actor, timestamp, revision ID, content hash, and request ID for every committed domain transition.

6 SCOPE BOUNDARIES & VERIFICATION MATRIX

In Scope (Guaranteed)
- ✓ Immutable revision tree & copy-on-write
- ✓ SHA-256 cryptographic content hashes
- ✓ Row-locked scheduling transaction
- ✓ Auto-revocation on post-approval edit
- ✓ Local SQLite outbox & idempotent sync
- ✓ Append-only audit & runbook queries

Out of Scope (Honest Limits)
- ✗ Ad delivery / real platform publishing
- ✗ Creative asset rendering & storage
- ✗ Billing, payments & reconciliation
- ✗ Push notifications / email delivery
- ✗ SSO / enterprise multi-tenancy
- ✗ Dual approval (deferred to ADR-001)

S1: Timed Journey <2 min walkthrough	**S2: Zero Invalid Schedules** DB invariant assert
S3: Exactly-Once Effect Idempotency stress test	**S4: Process-Kill Safety** SQLite outbox re-sync
S5: Complete Auditability 100% transitions attributed	**S6: Explicit Conflict** 409 on stale revision review